JAVA DOCS

Microservices — Architecture Deep Dive

Decomposition, Bounded Contexts, Communication Styles & Migration Strategy

Java Agentic AI — Knowledge Base Reference

This document covers microservices as an architectural discipline — decomposition strategy, domain boundaries, and organizational implications. For Spring-specific implementation details (Eureka, Spring Cloud Gateway, Resilience4j), see Microservices.pdf in spring_docs/.

1. What Actually Defines a Microservice?

A microservice is an independently deployable service, owning its own data, built around a specific business capability. The defining property isn't size (there's no line-count threshold) — it's **independent deployability**: you can change and release one service without having to coordinate a synchronized deployment of any other service.

2. Decomposing by Business Capability, Not Technical Layer

A common early mistake is decomposing services by technical layer (a 'UI service,' a 'business logic service,' a 'database service') — this actually recreates a distributed monolith, since any single business change still requires touching multiple services in lockstep. The far more effective approach, borrowed from Domain-Driven Design, is decomposing along **bounded contexts** — cohesive areas of business capability (Order Management, Inventory, Payments, Customer Identity) each with their own data and language.

Decomposition style	Example split	Outcome
By technical layer (anti-pattern)	UI service, business-logic service, data-access service	Every feature change touches multiple services — c
By business capability (recommended)	Order service, Inventory service, Payment service, Shipping service	Most feature changes are contained within a single

3. Bounded Contexts and Ubiquitous Language

A bounded context is a boundary within which a specific business model and vocabulary applies consistently. The same real-world word can mean different things in different contexts — 'Customer' in the Sales context might mean a lead with contact preferences, while 'Customer' in the Billing context means a legal entity with payment terms and tax IDs. Trying to force one unified 'Customer' model across every service is a common cause of accidental coupling — each bounded context should be free to model the same real-world concept differently, translated at the boundary where contexts interact.

4. Strangler Fig Pattern — Migrating a Monolith Incrementally

Named after a strangler fig vine that gradually envelops and eventually replaces its host tree, this pattern migrates a monolith to microservices incrementally rather than through a risky 'big bang' rewrite. New functionality (and gradually, extracted existing functionality) is built as new services, while a routing layer (often an API Gateway or reverse proxy) directs traffic to either the old monolith or the new service depending on what's been migrated, until eventually the monolith handles nothing and can be retired.

- Identify a well-bounded piece of functionality with a clear seam (e.g. a self-contained module with few cross-cutting dependencies) to extract first.

- Stand up the new service, routing a subset of traffic (or a specific endpoint) to it via the gateway/proxy.
- Keep the monolith's equivalent code as a fallback until the new service is proven under real production load.
- Repeat, service by service, rather than attempting a full simultaneous rewrite — dramatically reducing the risk of any single migration step.

5. Data Ownership and Consistency

Each service owns its data exclusively — no other service should read from or write directly to another service's database. This enforces the service boundary at the data layer, not just the code layer, preventing the classic failure mode where 'independent' services are secretly coupled through a shared table.

5.1 Handling Cross-Service Transactions — the Saga Pattern

Since a traditional ACID transaction can't span multiple databases, a Saga breaks a business transaction into a sequence of local transactions, each in one service, with compensating actions to reverse prior steps if a later step fails.

```
Order Service:     creates order (PENDING) -> publishes OrderCreated
Inventory Service: reserves stock -> publishes StockReserved (or StockReservationFailed)
Payment Service:     charges customer -> publishes PaymentCompleted (or PaymentFailed)
Order Service:        on PaymentFailed -> compensating action: cancel order, release
reserved stock
```

Saga style	How it works	Trade-off
Choreography	Each service listens for relevant events and reacts independently, no central coordinator	Simple for few steps, but harder to visualize/debug as st
Orchestration	A central orchestrator explicitly tells each service what to do next and tracks overall progress	Easier to reason about complex flows, but the orchestrato

6. API Composition and CQRS

A query that needs data spanning multiple services (e.g. 'show a customer's order history with current shipping status') can't be answered by one service's database alone. Two common approaches: **API composition** (a composing service calls each relevant service and merges results at request time) and **CQRS with a read-optimized view** (a separate service maintains its own denormalized, pre-joined view built from events published by the owning services, queried directly without cross-service calls at read time).

> ***Note:*** *API composition is simpler to build but adds request-time latency and coupling to every dependency's availability. A CQRS read model trades eventual consistency and additional infrastructure for much faster, more resilient reads at scale.*

7. Organizational Alignment — Conway's Law

Conway's Law observes that any organization designing a system will produce a design whose structure mirrors the organization's own communication structure. In practice: if service boundaries don't align with team boundaries, every cross-service change requires cross-team coordination, undermining the independent-deployability benefit microservices are meant to provide. This is why many organizations deliberately apply the 'inverse Conway maneuver' — structuring teams around the desired service boundaries first, rather than the reverse.

8. Trade-offs: When NOT to Use Microservices

- Small teams/organizations — the operational overhead (multiple deployments, distributed tracing, network reliability, data consistency) often outweighs any benefit until an organization's scale or team count justifies the complexity.
- Poorly understood domain boundaries — decomposing before the business domain is well understood risks drawing service boundaries in the wrong place, requiring expensive re-splitting later; starting with a well-modularized monolith and extracting services once boundaries stabilize is a common, pragmatic path (sometimes phrased as 'monolith-first').
- Tight latency/consistency requirements — some problems genuinely need strong, immediate consistency across the whole operation, which is much harder to guarantee across network boundaries than within a single process/database transaction.

9. Observability Requirements Unique to Microservices

A single logical user request can fan out across many services — without deliberate tooling, diagnosing a failure or slowdown becomes far harder than in a monolith where everything is one process and one log stream.

Concern	Why it's harder	Common tooling
Tracing	One request touches many services across the network	Distributed tracing with a propagated trace ID (Zipkin, Jaeger)
Logging	Logs are scattered across many independently-scaled instances	Centralized log aggregation (ELK stack, Loki)
Metrics	Health of the overall system depends on many independent services	Per-service metrics scraped centrally (Prometheus + Grafana)
Failure isolation	One slow/failing service can cascade to callers	Circuit breakers, bulkheads, timeouts at every service bounda

10. Quick Interview Recap

- What actually defines a microservice? Independent deployability around a business capability — not a specific size or line count.
- Why decompose by business capability instead of technical layer? Decomposing by layer forces cross-service coordination for nearly every feature change, effectively recreating a distributed monolith with extra network overhead.
- What is the Strangler Fig pattern used for? Incrementally migrating a monolith to microservices by routing traffic to new services piece by piece, avoiding a risky all-at-once rewrite.

- What is Conway's Law and why does it matter for microservices? System architecture tends to mirror organizational communication structure; misaligned team and service boundaries undermine the independent-deployability benefit microservices are meant to deliver.
- When would you NOT recommend microservices? Small teams, poorly understood domain boundaries, or workloads with strict cross-entity consistency/latency requirements that are much simpler to satisfy within a single process and database.